



Technical Report

Skills Path Architecture for Agile Development *Set the definition of done for a skill in development*

SUBMITTED BY
Maamoun Youssef

Jira Clone
2026-06-01

Abstract

This report describes how the development skills directory in this Salesforce/LWC project is shaped to automate the Development phase of the Agile SDLC for the developer (create a definition of done for task development in agile framework), while adapting to the project's architecture and rules. It is organized around the three concrete challenges the architecture has to solve — reliability, consistency, and maintainability — each illustrated with a project-specific example. The orchestrator owns classification only; each downstream task-skill owns its disciplined interview and code emission, and shared sub-steps are factored into task-skill/shared/ to avoid a cross-skill dependency mesh.

1. The goal

This skill set is not a grab-bag of recipes. It targets one intersection of the SDLC and one role, under one set of constraints.

Question	Answer
Which SDLC phase does this automate?	The Development phase of Agile.
Who uses it?	The developer.
For what?	To implement a task (the bug flow is reserved for later).
Constrained by what?	The project's architecture and rules — LWC architecture, Apex bulkification, soft-delete filter, required fields, controller → Service → APIResponse layering.

Restated:

Build a skill that automates the Development phase of the Agile SDLC for the developer, that adapts to this project's architecture, rules, and framework, and that delivers reliability, consistency, and maintainability without architectural overhead.

The rest of this document is the architecture that meets that goal, organized by the three challenges it has to solve.

2. The three challenges (with examples)

2.1 Reliability — the right skill must fire even when the user didn't name it

Problem. A skill only fires if the model picks it. If the user's prompt doesn't name a downstream skill, it gets silently skipped.

Solution. Make the call explicit inside the calling skill, so it cannot be missed regardless of wording.

Example. The user says "create a new parent LWC" — nothing about CSS. The parent task-skill itself names the CSS handoff as a fixed step:

```
create-new-parent-lwc-component.md (line 207)  
Step 4 – Optionally apply lwc-css-design
```

2.2 Consistency — same intent, same path, every time

Problem. With N skills directly addressable, the user's wording decides which one fires. Same intent, different phrasings, different paths.

Solution. One entry point — the orchestrator — and an AskUserQuestion flow that turns the prompt into a fixed set of answers, so the user's intent is captured with 100% certainty.

Example. Any dev request enters agile-development-orchestrator-skill.md and answers two AskUserQuestion calls: bug vs. task, then A/B/C + parent/child. Three different phrasings of "build a feature on a parent LWC" all land on the same row:

```
B – create new functionality / parent →  
add-interactive-with-data-persistence-functionality-in-parent.md
```

2.3 Maintainability — a new task can't fan out edits across every other task

Problem. When several task-skills need the same sub-step, in-lining it in each one means every change has to touch every copy.

Solution. Extract the shared sub-step into one file under task-skill/shared/ and have each task-skill call it.

Example. Both add-interactive-with-data-persistence-functionality-in-parent.md and lwc-parent-event-handler-generator.md need to resolve "which Apex method backs this?". Instead of asking that interview manually in each file, both call shared/apex-method-resolution.md — the three-branch interview (existing-known / existing-find / create-new) lives in exactly one file.

3. Adaptation to project architecture and rules

The orchestrator does not invent rules — it routes work into skills that already encode the project's rules. The chain of constraints, as they fire during a typical "create a new parent LWC" iteration:

- Orchestrator — classifies as task / option A / parent.

- create-new-parent-lwc-component — Step 0 mandates lwc-architecture end-to-end (page vs. child, principal state shape, localStorage keys, Apex methods, event names, sidecar utils / validator, meta.xml exposure).
- Per-iteration sub-skill (A → functionality, B → event handler) runs its full disciplined interview, then hands off to the shared Apex-method-resolution protocol when controller methods need to be resolved.
- Final Verification — the project-wide MANDATORY skills declared in CLAUDE.md are checked: bulkified Apex (apex-bulk-soql), soft-delete filter (soql-exclude-deleted), required fields per OBJECT_VALIDATION_LWC_APEX.md (object-required-fields), thin controller → Service → APIResponse layering (lwc-architecture).
- Optional CSS handoff — Step 4 asks whether to apply lwc-css-design so the new parent drops in next to manageBacklog / manageWorkflow without visual tuning.

Every project-specific rule has exactly one home, and the orchestrator guarantees the rules are REACHED — not left to the model to remember.

4. Current file layout (for reference)

```
force-app/skills/development/
├── agile-development-orchestrator-skill.md    ← single entry
├── architecture/
│   ├── lwc-architecture.md
│   └── lwc-css-design.md
├── contract/
│   ├── object-required-fields-skill.md
│   └── soql-exclude-deleted-skill.md
├── guard/
│   ├── apex-governor-limit-guard.md
│   ├── lwc-apex-loading.md
│   └── lwc-error-handling-skill.md
├── performance/
│   ├── apex-bulk-soql.md
│   └── apex-method-monitor.md
├── task-skill/
│   ├── add-functionality-in-child-lwc-component.md
│   ├── add-interactive-with-data-persistence-functionality-in-parent.md
│   ├── create-new-child-lwc-component.md
│   ├── create-new-parent-lwc-component.md
│   ├── lwc-parent-event-handler-generator.md
│   └── shared/
│       └── apex-method-resolution.md
```